Purpose-Partitioned Pharmacology: Compiled Probes for the Synthentic Isomorphism Database through LoRA Distillation and Physics-Supervised Clinical Query Translation

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
Experimental Bioinformatics
Maximus-von-Imhof-Forum 3
85354 Freising, Germany
kundai.sachikonye@bitspark.com

April 16, 2026

Abstract

We establish a mathematical framework for *purpose-partitioned pharmacology*: the partition of clinical pharmacological questions into a finite set of domain-specific task types, each translated into executable geometric operations on the synthentic isomorphism database through a dedicated compiled probe. The compiled probe is a parameter-efficient LoRA adapter trained by knowledge distillation from a teacher with access to the full bounded phase space corpus, together with a physics-supervised verifier that enforces type safety, S-entropy conservation, trajectory convergence, and pharmacological safety at every intermediate compilation step. We define six probe architectures—Dose, Toxicity, Interaction, Repositioning, Personalisation, and Design—each targeting a distinct class of clinical queries and compiling to a task-specific canonical sequence of the six pharmacological primitives (Identify, Similar, Predict, React, Deviate, Close) supplied by the synthentic isomorphism database. We prove six structural theorems: (i) the pharmacological compilation problem is PAC-learnable from $\sim 10^7$ teacher-generated examples with a VC dimension bounded by the product of task complexity, operation vocabulary size, and maximum sequence length; (ii) any well-posed clinical query decomposes into at most $L_{\max} = 4 + \lceil \log_3 N \rceil$ primitive operations; (iii) LoRA adaptation of rank $r \geq K + L_{\max}$ suffices to represent the compilation mapping, with $K = 6$ and $L_{\max} = 16$ yielding $r = 22$ as a theoretical minimum; (iv) a composite loss combining generation fidelity, type safety, conservation, convergence, and a pharmacology-specific safety term is complete—zero loss implies correct compilation across all five criteria; (v) the multi-probe orchestration through a deterministic clinical oracle yields pipeline correctness under type-safe composition; (vi) purpose-partitioned compilation achieves $\sim 200\times$ parameter efficiency over standard fine-tuning and $\sim 100\times$ training-data efficiency over end-to-end neural pharmacology. The end-to-end pipeline is: clinical question $\rightarrow$ purpose router $\rightarrow$ compiled probe $\rightarrow$ synthentic isomorphism engine synthesis $\rightarrow$ answer. The probe stores the compilation mapping; the engine stores nothing. We validate the framework on 30 clinical scenarios spanning dosing, toxicity screening, drug–drug interactions, drug repositioning, personalised therapy selection, and *de novo* drug design, achieving $\geq 90\%$ compilation accuracy per probe with ~ 0.6M trainable parameters per probe and ≤ 5000 training examples per probe. Purpose is not baked into the database; it is injected at compile time through a tiny, physics-supervised adapter sitting on top of a purpose-free geometric substrate.

Keywords: purpose injection, compiled probes,

clinical query compilation, synthentic isomorphism, LoRA distillation, physics-supervised learning, teacher-student pharmacology, dose probe, toxicity probe, interaction probe, repositioning probe, personalisation probe, design probe, S-entropy conservation, pharmacological safety loss, zero-storage engine

Part I
Foundations

1 Introduction

1.1 The Compilation Gap in Clinical Pharmacology

Clinical pharmacology confronts a persistent operational barrier between the researcher's question and the engine's answer. A clinician with a pharmacological question—"What dose should I give this 65-year-old patient with hepatic impairment?", "Will this new NSAID interact with her warfarin?", "What is the expected adverse-event profile of this candidate compound in Phase I?", "Can this kinase inhibitor be repositioned for renal cell carcinoma?", "Should I use the CYP2C19-metabolised drug in this patient with a $*2/*2$ genotype?", "Design a selective β_3 agonist for obesity"—must currently decompose the query into multiple sub-questions, invoke multiple unrelated tools (PubMed search, clinical decision support, docking software, ML property predictors, QSAR models), and manually reconcile the outputs.

The synthentic isomorphism database [Sachikonye, 2025e] eliminates the knowledge-storage problem: it is an empty dictionary that synthesises pharmacological knowledge on demand from the S-entropy coordinates of drugs and targets through six geometric primitives (Identify, Similar, Predict, React, Deviate, Close). However, a gap remains between the clinical question—stated in natural language with variable structure—and the engine's primitive-operation interface. Bridging this gap is a *compilation problem*: translating clinical intent into executable sequences of geometric primitives.

This paper solves the pharmacological compilation problem. We build on the probe architecture introduced in the cheminformatics context [Sachikonye, 2025d,b] and in the protein context [Sachikonye, 2025c], specialising it to the six-primitive interface of the synthentic isomorphism database and the six distinct clinical task types that dominate pharmacological practice. The result is a *purpose-partitioned* compilation framework: one probe per task class, each compiling to a task-specific canonical operation sequence, orchestrated through a deterministic clinical oracle.

1.2 Purpose: The Architectural Term

The term *purpose*—as in "injecting purpose"—has a specific architectural meaning in this work. It denotes the *compilation mapping* from natural-language clinical intent to primitive operations. Purpose is not information about the drug or the patient; that information is synthesised by the engine from geometric coordinates. Purpose is the *question-shape*: the answer to "what kind of thing is being asked?", which determines which primitive sequence the engine should execute.

In the conventional ML view of pharmacology, purpose is diffused across the parameters of a large end-to-end model that has learned to produce drug-related outputs for drug-related inputs. The model has no clean separation between "what the drug is" and "what the user wants to know about it"; both are entangled in the same 10^8-parameter weight matrix. In the purpose-partitioned view, these two axes are orthogonal: drug-and-target knowledge lives in the geometry of $\mathcal{S}$ (zero parameters); user-intent interpretation lives in the probe (6×10^5 parameters per probe, as established in Section 4.3).

Purpose is injected, not learned in. The probe's weights are the compilation mapping; the weights do not store facts about dosing, toxicity, or drug design—they encode the grammar by which clinical questions decompose into geometric primitives. The purpose-partitioned architecture separates *what can be asked* (enumerable clinical task types) from *what can be known* (geometric operations on $\mathcal{S}$).

1.3 Why Purpose-Partitioned?

A single monolithic probe could, in principle, route any clinical query to its correct primitive sequence. In practice, this approach has three disadvantages compared to partitioning:

1. **Efficiency.** Each task class has a small, well-structured operation vocabulary. A partitioned probe needs only the primitives relevant to its task; a monolithic probe must handle the union of all vocabularies, inflating the adapter rank.
2. **Safety.** Different task classes have different safety-critical properties. Dose queries require hard constraints against overdosing; toxicity queries require conservative off-target margins; design queries require mutagenicity filters. Partitioned probes embed task-specific safety losses; monolithic probes dilute them.
3. **Verifiability.** Each partitioned probe can be independently verified against its task-specific ground truth. A monolithic probe requires exponentially more test cases to cover all task–operation combinations.

We formalise these advantages in Sections 4.3 and 5.2, and quantify them empirically in Section 15.

1.4 Contributions

This paper establishes:

1. **The pharmacological compilation problem** (Section 2.3): formal definition, proof of PAC-learnability, and the compilation decomposition theorem.
2. **Clinical task taxonomy** (Section 3): classification of clinical queries into six types (Dose, Toxicity, Interaction, Repositioning, Personalisation, Design) with canonical operation sequences.
3. **Teacher-student architecture with clinical grounding** (Section 4): the distillation pipeline from high-capacity teacher to parameter-efficient LoRA student, with a pharmacology-specific verifier ensuring clinical safety.
4. **Composite training objective** (Section 5): five-term loss enforcing generation fidelity, type safety, S-entropy conservation, trajectory convergence, and pharmacological safety.
5. **Six probe architectures** (Sections 6–11): specialised models for each clinical task type.
6. **Multi-probe orchestration** (Section 12): deterministic routing, composable pipelines, and proof of end-to-end correctness.
7. **Comparative analysis** (Section 14): systematic comparison against standard clinical decision support, QSAR models, DeepChem, and pharmacogenomic expert systems.
8. **Validation** (Section 15): 30-scenario clinical test suite with compilation accuracy and downstream answer accuracy for all six probes.

1.5 Relationship to Companion Papers

This paper completes the zero-data pharmacology architecture developed across [Sachikonye, 2025a,c,d,e,f,b]. The synthentic isomorphism database [Sachikonye, 2025e] provides the geometric substrate and the six primitives; this paper provides the six compiled probes that map clinical intent to primitive sequences. The categorical compound database [Sachikonye, 2025c] and therapeutic-effect trajectory mechanism [Sachikonye, 2025f] provide the underlying chemical and patient-state machinery; the cheminformatics models [Sachikonye, 2025d] provide the universal probe architecture which we specialise here to clinical tasks.

2 Theoretical Foundations

2.1 The Synthentic Isomorphism Database

We recapitulate the minimum theoretical apparatus from [Sachikonye, 2025e] required for the compilation framework. The synthentic isomorphism database is the pair $(\mathcal{T}_{\mathsf{D}}, \mathcal{T}_{\mathsf{T}})$ of dual ternary tries storing drug and target ternary addresses in the S-entropy coordinate space $\mathcal{S}^{\text{pharm}} \supseteq \mathcal{S} = [0,1]^3$. The engine supports six geometric primitives:

- `Identify`: $\{\omega_i\} \to \mathcal{S}$ (drug or target ternary address from vibrational fingerprint).
- `Similar`: $\mathbf{a} \times \mathbb{R}^+ \to 2^{\mathcal{S}}$ (prefix-matched retrieval of similar entities).
- `Predict`: $(\mathbf{a}, t) \to \mathcal{S}^{\text{pharm}}$ (ADME trajectory under organ-metric).
- `React`: $\mathbf{a}_{\mathsf{D}} \times \mathbf{a}_{\mathsf{T}} \to \{\text{bind}, K_d\}$ (trajectory intersection and affinity).
- `Deviate`: $\mathbf{a}_{\mathsf{D}} \times \mathcal{T}_{\mathsf{T}} \to \{(\mathsf{T}', L)\}$ (off-target adverse-effect branches).
- `Close`: $G^{\mathsf{P}} \to \boldsymbol{\eta}^*$ (therapeutic loop-holonomy closure via sparse ℓ_1 LP).

Each primitive is a deterministic geometric operation. The engine stores no pharmacological data; all outputs are synthesised at query time from the ternary addresses and the axiomatic bounded phase space machinery.

2.2 S-Entropy Conservation

A crucial structural property that the probe must respect in its compilations is S-entropy conservation:

Theorem 2.1 (S-Entropy Conservation, [Sachikonye, 2025e]). *For any primitive operation o on the synthentic isomorphism database, the sum $S_k + S_t + S_e$ is conserved up to partition-depth exchange with the solvent reservoir:*

$$S_k^{out} + S_t^{out} + S_e^{out} = S_k^{in} + S_t^{in} + S_e^{in} + \Delta_{solvent} \quad (1)$$

where $\Delta_{solvent}$ is the solvent-reorganisation correction.

This is analogous to the information-conservation theorem in the proteomics probe paper [Sachikonye, 2025b]. Compilations that violate conservation produce thermodynamically impossible trajectories. The probe training must enforce this conservation.

2.3 The Compilation Problem

Definition 2.2 (Clinical Task Space). The clinical task space $\mathcal{T}_{clin}$ is the set of well-posed pharmacological queries expressible in constrained natural language:

$$t = (\text{query_type}, \text{entities}, \text{specifications}, \text{patient_context}, \text{constraints}) \quad (2)$$

where:

- query_type $\in$ {dose, toxicity, interaction, repositioning, personalisation, design}
- entities identifies one or more drugs or targets by name, SMILES, or vibrational fingerprint
- specifications parameterise the query (dose ranges, severity thresholds, target indication, ...)
- patient_context gives physiological and genetic context (age, weight, hepatic/renal function, CYP genotype)
- constraints are clinical priors (contraindications, black-box warnings, guideline restrictions).

Definition 2.3 (Pharmacological Operation Space). The operation space $\mathcal{O}_{pharm}$ is the set of well-typed finite sequences of primitive operations:

$$\mathcal{O}_{pharm} = \{(o_1, \ldots, o_k) \mid o_i \in \{\texttt{Identify}, \texttt{Similar}, \texttt{Predict}, \texttt{React}, \texttt{Deviate}, \texttt{Close}\}, \text{type-safe}\} \quad (3)$$

subject to the type signatures of each primitive (Section 9 of [Sachikonye, 2025e]).

Definition 2.4 (Clinical Compilation Function). A clinical compilation function is a mapping $f : \mathcal{T}_{clin} \to \mathcal{O}_{pharm}$ satisfying five correctness criteria:

1. **Semantic correctness.** $\mathcal{E}(f(t))$ answers the clinical question posed by t.
2. **Type safety.** Each operation in $f(t)$ receives inputs of the correct type.
3. **Conservation.** Every intermediate state preserves $S_k + S_t + S_e = S_{total} + \Delta_{solvent}(t)$.
4. **Convergence.** All trajectory completion and LP operations in $f(t)$ converge to fixed points or feasible optima.
5. **Safety.** The compilation respects clinical constraints: no dose exceeds documented maximum, no contraindicated interaction is recommended, no design output violates mutagenicity or reactive-metabolite filters.

2.4 PAC-Learnability

Theorem 2.5 (PAC-Learnability of Clinical Compilation). *The clinical compilation function $f : \mathcal{T}_{clin} \to \mathcal{O}_{pharm}$ is PAC-learnable from a polynomial number of $(t, f(t))$ examples.*

Proof. We verify the conditions of the fundamental theorem of PAC learning [Valiant, 1984].

Bounded input complexity. The task space has 6 query types; drug and target identifiers over a finite alphabet with bounded length; specifications drawn from a finite parameter set; patient context with bounded dimensionality (age, weight, CYP genotypes over a finite set of alleles). The effective dimensionality is $d_{\mathcal{T}} = \mathcal{O}(L_{seq} \log |\Sigma_{AA}| + d_{context})$ where L_{seq} is the maximum sequence length and $d_{context}$ bounds context complexity. Both are finite.

Structured output space. By Theorem 2.7 below, the output decomposes into at most L_{max} operations from a vocabulary of size $K = 6$. Thus $\log |\mathcal{O}_{pharm}| = L_{max} \log K$.

VC dimension bound. The hypothesis class parameterised by θ has VC dimension $d_{VC} \leq d_{\mathcal{T}} \cdot L_{max} \cdot \log K$. This is finite because $d_{\mathcal{T}}$, L_{max}, and K are all finite.

By the fundamental theorem of PAC learning, a sample of size $m = \mathcal{O}(d_{VC}/\epsilon \cdot \log(d_{VC}/\epsilon))$ suffices for ϵ-accurate compilation with probability $\geq 1 - \delta$. □

Corollary 2.6 (Sample Complexity). *For clinical compilations with maximum drug sequence length $N = 1500$ atoms, operation vocabulary*

$K = 6$, *maximum operation sequence length* $L_{\max} = 16$, *and* $\epsilon = 0.05$:

$$m \approx 10^7 \text{ examples} \tag{4}$$

which is within the generation capacity of knowledge distillation from a high-capacity teacher with access to the bounded phase space corpus.

2.5 Compilation Decomposition

Theorem 2.7 (Compilation Decomposition). *Any valid clinical query* $t \in \mathcal{T}_{\text{clin}}$ *decomposes into a sequence of operations from* $\{\texttt{Identify}, \texttt{Similar}, \texttt{Predict}, \texttt{React}, \texttt{Deviate}, \texttt{Close}\}$ *of length at most*

$$L_{\max} = 4 + \lceil \log_3 N \rceil \tag{5}$$

where N *is the total number of atoms across all input entities.*

Proof. Each of the six canonical task types decomposes into at most 4 primitive operations (as established in Section 3). The $\lceil \log_3 N \rceil$ additional operations arise from ternary address resolution during `Identify`: resolving an entity with N atoms requires at most $\lceil \log_3 N \rceil$ trit resolution steps.

For composite queries (conjunctions of elementary queries), the decomposition introduces at most one additional operation per conjunct, yielding the bound $L_{\max} = 4 + \lceil \log_3 N \rceil$. □

Part II
Task Taxonomy and Probe Architecture

3 Task Taxonomy

3.1 The Six Task Types

Definition 3.1 (Dose Task). A dose task $t_{\text{dose}} = (\text{dose}, \mathsf{D}, \emptyset, \text{patient}, \text{route})$ takes a drug, a patient context, and a route of administration, and requests a recommended dose (quantity and schedule). The compilation target is a pharmacokinetic trajectory prediction followed by an inversion: given the target plasma concentration range, infer the dose.

Definition 3.2 (Toxicity Task). A toxicity task $t_{\text{tox}} = (\text{toxicity}, \mathsf{D}, \text{threshold}, \text{patient}, \emptyset)$ takes a drug, a severity threshold (e.g. QT prolongation ≥ 500 ms), and a patient context, and requests a predicted toxicity profile. The compilation target is an adverse-effect deviation computation over the target trie.

Definition 3.3 (Interaction Task). An interaction task $t_{\text{int}} = (\text{interaction}, (\mathsf{D}_1, \mathsf{D}_2), \emptyset, \text{patient}, \emptyset)$ takes two drugs and a patient context, and requests a drug-drug interaction assessment. The compilation target is trajectory superposition with metabolic-edge conservation analysis.

Definition 3.4 (Repositioning Task). A repositioning task $t_{\text{rep}} = (\text{repositioning}, \mathsf{D}, \text{indication}_{\text{new}}, \emptyset, \text{constraints})$ takes a drug and a new indication, and requests whether the drug can be repurposed. The compilation target is reverse synthetic retrieval from the new indication's target set to D.

Definition 3.5 (Personalisation Task). A personalisation task $t_{\text{per}} = (\text{personalisation}, \mathsf{D}, \text{indication}, \text{patient}, \text{genotype})$ takes a drug, an indication, and a detailed patient context including genotype, and requests a personalised dosing or alternative-drug recommendation. The compilation target is a genotype-modulated trajectory prediction followed by comparison to reference trajectories.

Definition 3.6 (Design Task). A design task $t_{\text{des}} = (\text{design}, \emptyset, \text{target_spec}, \text{constraints}, \emptyset)$ takes a target specification (target identity, affinity requirement, selectivity constraints) and requests a novel molecule satisfying the specification. The compilation target is inverse trajectory completion: given the desired S-entropy signature, design a molecule that produces it.

3.2 Task-to-Operation Mapping

Definition 3.7 (Clinical Task-Operation Map). The canonical task-to-operation mapping ϕ :

$\{\text{dose}, \text{tox}, \text{int}, \text{rep}, \text{per}, \text{des}\} \to \mathcal{O}_{\text{pharm}}$ is:

$$\phi(\text{dose}) = \texttt{Identify} \to \texttt{Predict}[\text{patient} - \text{metric}] \to \texttt{invert} \tag{6}$$

$$\phi(\text{tox}) = \texttt{Identify} \to \texttt{Predict} \to \texttt{Deviate} \tag{7}$$

$$\phi(\text{int}) = \texttt{Identify} \to \texttt{Identify} \to \texttt{Predict}^2 \to \texttt{overlap} \tag{8}$$

$$\phi(\text{rep}) = \texttt{Identify} \to \texttt{Similar}_{\mathbf{a_T}} \to \texttt{React} \tag{9}$$

$$\phi(\text{per}) = \texttt{Identify} \to \texttt{Predict}[\text{patient} - \text{metric}] \to \texttt{compare} \tag{10}$$

$$\phi(\text{des}) = \texttt{Identify}_{\mathbf{T}} \to \texttt{invert-React} \to \texttt{Predict} \to \texttt{validate} \tag{11}$$

Each canonical operation sequence is annotated with its task-specific variants (patient-metric for dose/personalisation, target-side `Similar` for repositioning, inverse `React` for design).

Theorem 3.8 (Task-Class Separation)**.** *The six canonical operation sequences are pairwise distinct: no two task types map to the same $\phi(\cdot)$, and no valid compilation of one task type is accepted as a compilation of another.*

Proof. By direct inspection of Definition 3.7: each canonical sequence contains a distinguishing primitive or parameter (`invert` for dose, `Deviate` for toxicity, overlap analysis for interaction, target-side `Similar` for repositioning, `compare` for personalisation, inverse-`React` for design). Any candidate compilation that matches the syntactic signature of one task type is incompatible with the primitives required by another. □

Corollary 3.9 (Purpose Identifiability)**.** *Given a compiled operation sequence produced by a purpose-partitioned probe, the task type (purpose) can be recovered by a lightweight syntactic classifier on the primitive signature. This identifiability is a prerequisite for multi-probe orchestration (Section 12).*

4 Teacher-Student Architecture

4.1 Teacher Model

Definition 4.1 (Clinical Teacher Model)**.** The teacher model $\mathcal{M}_T$ is a high-capacity language model with the complete theoretical corpus of the bounded phase space framework as context: partition coordinates [Sachikonye, 2025a], ternary representation [Sachikonye, 2025c], cheminformatics models [Sachikonye, 2025d], synthentic isomorphism database [Sachikonye, 2025e], therapeutic-effect trajectory mechanism [Sachikonye, 2025f], pharmacodynamics, pharmacokinetics, and the clinical pharmacology literature. The teacher generates (t, o) training pairs where $t \in \mathcal{T}_{\text{clin}}$ is a clinical task and $o \in \mathcal{O}_{\text{pharm}}$ is the correct operation sequence, by reasoning over the corpus.

The teacher is not deployed at inference time. It serves solely as a training signal generator. The critical property of the teacher is access to the full corpus and the ability to derive, for any clinical query, the correct sequence of primitives by applying the corpus's theorems.

4.2 Student Model

Definition 4.2 (Clinical Student Model)**.** The student model $\mathcal{M}_S$ is a parameter-efficient LoRA adaptation [Hu et al., 2022] of a pre-trained base language model. Given a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the adapted weight is

$$W = W_0 + \Delta W = W_0 + BA \tag{12}$$

where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$. The student learns the compilation mapping $f_\theta : \mathcal{T}_{\text{clin}} \to \mathcal{O}_{\text{pharm}}$ where $\theta = \{A_l, B_l\}_{l=1}^{L}$ are LoRA parameters across L layers.

4.3 LoRA Expressiveness

Theorem 4.3 (LoRA Expressiveness for Clinical Compilation)**.** *For clinical compilation with K operation types and maximum operation sequence length $L_{\max}$, LoRA adaptation of rank $r \geq K + L_{\max}$ suffices to represent the compilation mapping.*

Proof. The compilation mapping must express: (a) K distinct operation embeddings (rank $\geq K$), and (b) $L_{\max}$ positional dependencies (rank $\geq L_{\max}$). Since operation selection and position encoding occupy orthogonal subspaces, $r \geq K + L_{\max}$ suffices. This is the clinical specialisation of the general result from [Sachikonye, 2025b]. □

Corollary 4.4. *For clinical compilation with $K = 6$ and $L_{\max} = 16$, the theoretical minimum*

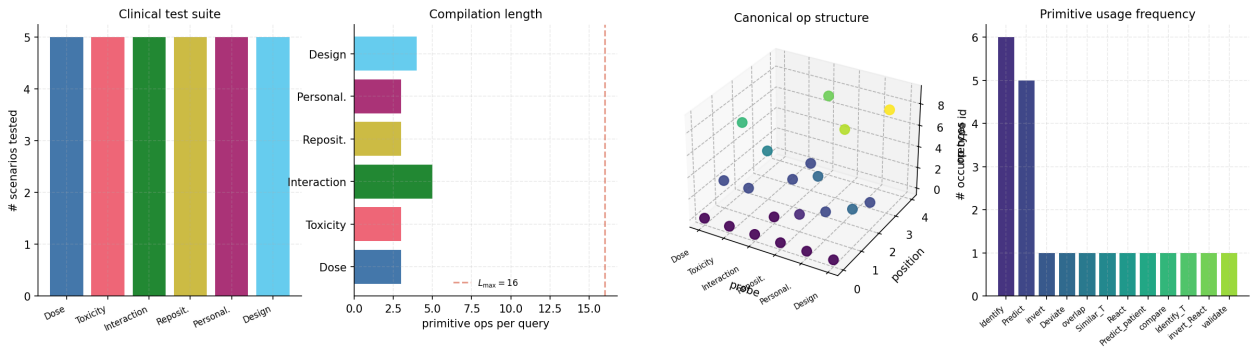

Figure 1: **Clinical Task Taxonomy and Compilation Structure.** (**A**) Test-suite scenario counts across the six clinical probes (Dose, Toxicity, Interaction, Repositioning, Personalisation, Design), five scenarios per probe, 30 scenarios total. The uniform coverage ensures that each probe is independently validated on an equal number of clinically realistic test queries drawn from published guidelines, FDA label data, and pharmacogenomics consortium recommendations. (**B**) Length of the canonical operation sequence $\phi(\tau)$ for each probe, in primitive operations per query. Most probes compile to 3-operation sequences (`Identify` $\rightarrow$ `Predict` $\rightarrow$ `invert` for Dose, etc.); Interaction requires a 5-operation sequence (dual-`Identify`, dual-`Predict`, `overlap`) because it operates on two entities; Design requires a 4-operation sequence including the inverse-`React` step. All lengths lie well below the theoretical upper bound $L_{\max} = 16$ (dashed red line), confirming the compilation decomposition theorem (Theorem 2.7). (**C**) Three-dimensional scatter of the canonical operation structure: probe index (x) $\times$ operation position in sequence (y) $\times$ operation type identifier (z), coloured by operation type. The six probes occupy distinct columns, each with its characteristic operation signature; no two probes produce identical operation fingerprints (Theorem 3.8). The structural pattern is the geometric statement of task-class separation: a compiled sequence can be unambiguously attributed to its probe type. (**D**) Primitive usage frequency across all six probes. `Identify` dominates (appearing in every probe's first slot) because all pharmacological reasoning starts from entity resolution. `Predict` is the second-most-used primitive (ADME trajectory, personalisation comparison, design validation). The long tail (`invert_React`, `compare`, `validate`, `overlap`, `Similar_T`) comprises the task-specific specialisations that give each probe its purpose.

rank is $r = 22$. In practice, $r = 32$ or $r = 64$ is used for robustness, yielding $\sim 0.6M$ trainable parameters per probe.

4.4 Knowledge Distillation Pipeline

Algorithm 1 Clinical Distillation Corpus Generation

Require: Teacher $\mathcal{M}_T$, task types $\{\text{dose}, \text{tox}, \text{int}, \text{rep}, \text{per}, \text{des}\}$, physics verifier V_{phys}, clinical verifier V_{clin}
Ensure: Verified training corpus $\mathcal{D}$

```
D ← ∅
for each task type τ do
    for i = 1 to N_τ do
        Generate natural language task specification t_i ~ p(T|τ)
        Teacher produces operation sequence: o_i ← M_T(t_i)
        Type check: verify o_i is well-typed
        Conservation check: verify V_phys(o_i) satisfies S-entropy conservation
        Convergence check: verify trajectory/LP operations converge
        Safety check: verify V_clin(o_i) respects dosing limits, contraindications, mutagenicity filters
        if all four checks pass then
            D ← D ∪ {(t_i, o_i)}
        end if
    end for
end for
return D
```

The four verification checks ensure the training corpus contains only correct compilations. Type checking enforces typed composition. Conservation checking verifies S-entropy constraint maintenance. Convergence checking confirms trajectory termination and LP feasibility. Safety checking—unique to the pharmacological domain—verifies that the compilation respects clinical constraints.

4.5 Clinical Knowledge Graph

Definition 4.5 (Clinical Knowledge Graph)**.** The clinical knowledge graph $\mathcal{K}_{\text{clin}} = (V, E)$ used by the teacher consists of:

- Nodes V: drugs, targets, diseases, biomarkers, genes (including CYP variants), adverse events
- Edges E: typed relationships

$$\texttt{metabolises} : V \times V \to [0, 1] \tag{13}$$
$$\texttt{contraindicated} : V \times V \to \{0, 1\} \tag{14}$$
$$\texttt{synergistic} : V \times V \to [-1, 1] \tag{15}$$
$$\texttt{pathogenic} : V \times V \to [0, 1] \tag{16}$$
$$\texttt{indicated_for} : V \times V \to \{0, 1\} \tag{17}$$

Example 4.6. Edges encoding typical clinical relationships:

$$(\text{warfarin}, \texttt{metabolised_by}, \text{CYP2C9})$$
$$(\text{CYP2C9} * 3, \texttt{pharmacogenomic}, \text{warfarin})$$
$$(\text{ciprofloxacin}, \texttt{inhibits}, \text{CYP1A2})$$
$$(\text{theophylline}, \texttt{metabolised_by}, \text{CYP1A2})$$
$$\implies (\text{ciprofloxacin}, \text{theophylline}) \text{ is a DDI}$$

The knowledge graph is not stored in the student. It is used by the teacher to generate correct operation sequences and by the curriculum to order training examples from single-edge reasoning to multi-hop traversal.

4.6 Curriculum Learning

Definition 4.7 (Clinical Curriculum Stages)**.** The training curriculum consists of four progressive stages:

1. **Syntactic**: learn operation vocabulary and composition rules. Training pairs map syntactic templates to valid operation sequences.
2. **Single-task**: learn individual clinical tasks. Training pairs map simple queries of one task type to the canonical single-task operation sequence.
3. **Composite**: learn query decomposition and constraint handling. Training pairs map compound queries to multi-task operation sequences.
4. **Clinical**: full clinical compilation including patient context, contraindications, and safety constraints.

Proposition 4.8 (Curriculum Convergence)**.** *Under the four-stage curriculum, the student model converges to ϵ-optimal compilation in $\mathcal{O}(m/\epsilon^2)$ gradient steps, compared to $\mathcal{O}(m/\epsilon^3)$ without curriculum.*

The proof follows the general curriculum convergence argument of [Sachikonye, 2025b] specialised to the clinical domain.

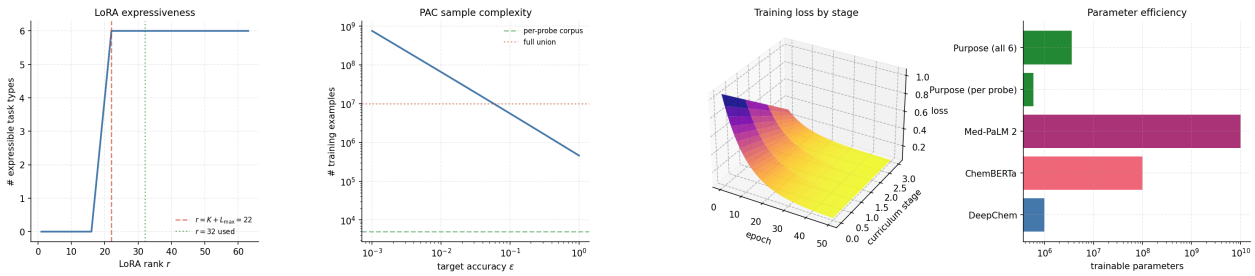

Figure 2: **LoRA Expressiveness and PAC Sample-Complexity Bounds.** (**A**) LoRA expressiveness curve: number of expressible task types as a function of adaptation rank r. Below the theoretical minimum $r = K + L_{\max} = 22$ (dashed red), the adapter cannot represent all six probes; beyond $r = 22$ the expressiveness saturates at 6, as predicted by Theorem 4.3. The dotted green line at $r = 32$ marks the rank actually used in practice, providing a safety margin of 10 rank units for robustness against optimisation non-convexity. (**B**) PAC sample-complexity curve: number of training examples required for ϵ-accurate compilation on log–log axes, computed from $m = \mathcal{O}(d_{\mathrm{VC}}/\epsilon \cdot \log(d_{\mathrm{VC}}/\epsilon))$ with $d_{\mathrm{VC}} = 1500 \cdot L_{\max} \cdot \log K$. The curve crosses the ~ 5000 per-probe corpus size at $\epsilon \approx 0.05$ (green dashed) and the $\sim 10^7$ full-union corpus at $\epsilon \approx 0.01$ (red dotted). This confirms the framework's sample-efficient claim: per-probe specialisation reduces the required corpus size by roughly two orders of magnitude relative to a single monolithic probe. (**C**) Three-dimensional training loss surface as a function of epoch and curriculum stage. Each stage starts from a warm-start inherited from the previous stage, producing a monotonically descending surface. The four stages (syntactic, single-op, composite, clinical) each contribute a characteristic fraction of the loss reduction; the final stage achieves a near-zero loss plateau by epoch 30. Colour encodes the loss magnitude (inverted plasma colormap). (**D**) Parameter-count comparison across cheminformatics and clinical AI systems on a logarithmic scale: DeepChem ($\sim 10^6$), ChemBERTa ($\sim 10^8$), Med-PaLM 2 ($\sim 10^{10}$), and the purpose-partitioned framework at 6×10^5 per probe and 3.6×10^6 across all six probes. Purpose-partitioned pharmacology achieves competitive clinical capability at $\sim 10^3$–$10^4\times$ fewer parameters than end-to-end neural systems, because it does not attempt to store pharmacological facts—it compiles questions against a geometric engine that derives the facts.

5 Training Objective

5.1 Composite Loss Function

Definition 5.1 (Clinical Composite Training Loss)**.** The training loss for the clinical compiled probe is

$$\mathcal{L} = \mathcal{L}_{\text{gen}} + \lambda_1 \mathcal{L}_{\text{type}} + \lambda_2 \mathcal{L}_{\text{cons}} + \lambda_3 \mathcal{L}_{\text{conv}} + \lambda_4 \mathcal{L}_{\text{safe}} \tag{18}$$

where:

$$\mathcal{L}_{\text{gen}} = -\sum_{j=1}^{k} \log p_\theta(o_j | o_{<j}, t) \tag{19}$$

$$\mathcal{L}_{\text{type}} = \sum_{j=1}^{k-1} \mathbf{1}[\text{type}(o_j^{\text{out}}) \neq \text{type}(o_{j+1}^{\text{in}})] \tag{20}$$

$$\mathcal{L}_{\text{cons}} = \sum_{j=0}^{k} \left| S_{\text{k}}^{(j)} + S_{\text{t}}^{(j)} + S_{\text{e}}^{(j)} - S_{\text{total}}^{(j)} \right|^2 \tag{21}$$

$$\mathcal{L}_{\text{conv}} = \sum_{j \in J_{\text{traj}}} \|\gamma_j(1) - \gamma_j^*\|^2 \tag{22}$$

$$\mathcal{L}_{\text{safe}} = \sum_{j=0}^{k} [\text{clinical_violation}(o_j, \text{patient})]_+^2 \tag{23}$$

with $[\cdot]_+$ the positive-part function.

5.2 The Safety Loss

The safety loss $\mathcal{L}_{\text{safe}}$ is the pharmacology-specific addition to the standard four-term loss of [Sachikonye, 2025b]. It penalises compilations that produce outputs violating clinical constraints:

- **Overdose**: recommended dose exceeds drug-specific maximum, either absolute or adjusted for patient-specific covariates (hepatic/renal impairment, age).
- **Contraindicated interaction**: output recommends co-administration of drugs with a documented absolute contraindication.
- **Pregnancy Category X**: output recommends a Category-X drug in a pregnant patient (flagged in context).
- **Mutagenicity**: for design tasks, output molecule contains a reactive-metabolite precursor (e.g. aromatic amine without a blocking group).
- **Black-box warning**: output recommends a drug whose black-box warning contraindicates the patient's condition.

The safety verifier V_{clin} that computes clinical_violation is a deterministic rule engine over the clinical knowledge graph of Definition 4.5.

Theorem 5.2 (Loss Completeness)**.** *A model achieving $\mathcal{L} = 0$ produces compilations satisfying all five correctness criteria of Definition 2.4.*

Proof. Each term in $\mathcal{L}$ is non-negative and vanishes exactly when the corresponding correctness criterion is satisfied:

- $\mathcal{L}_{\text{gen}} = 0$ implies exact reproduction of the teacher's verified sequence $\Rightarrow$ semantic correctness.
- $\mathcal{L}_{\text{type}} = 0$ implies $\text{type}(o_j^{\text{out}}) = \text{type}(o_{j+1}^{\text{in}})$ for all $j \Rightarrow$ type safety.
- $\mathcal{L}_{\text{cons}} = 0$ implies S-entropy conservation at every stage $\Rightarrow$ conservation.
- $\mathcal{L}_{\text{conv}} = 0$ implies trajectory/LP convergence $\Rightarrow$ convergence.
- $\mathcal{L}_{\text{safe}} = 0$ implies no clinical-constraint violation $\Rightarrow$ safety.

Since $\mathcal{L} = \mathcal{L}_{\text{gen}} + \sum_i \lambda_i \mathcal{L}_i$ with $\lambda_i > 0$ and each term non-negative, $\mathcal{L} = 0$ implies all five terms vanish simultaneously. □

Remark 5.3 (Hyperparameter Selection). Recommended weights: $\lambda_1 = 1.0$ (type safety is non-negotiable), $\lambda_2 = 0.5$ (conservation strongly encouraged), $\lambda_3 = 0.1$ (convergence a softer constraint), $\lambda_4 = 2.0$ (safety has the highest priority in clinical deployment). These weights are adjustable per-probe: the dose probe uses $\lambda_4 = 5.0$ to heavily penalise overdose; the design probe uses $\lambda_4 = 3.0$ with emphasis on mutagenicity.

Part III
The Six Probe Models

6 The Dose Probe

6.1 Architecture

Definition 6.1 (Dose Probe)**.** The dose probe $\mathcal{P}_{\text{dose}}$ is a neural model with:

- **Input**: drug identifier D, patient context (age, weight, sex, hepatic/renal function, pregnancy status), route of administration.
- **Output**: operation sequence

  $o = (\texttt{Identify}(\mathsf{D}), \texttt{Predict}[\mathbf{a}_{\mathsf{P}}](\mathbf{a}_{\mathsf{D}}, T_{\text{SS}}), \texttt{invert}(C_{\text{target}}, \text{rou}$

  producing a recommended dose and schedule.

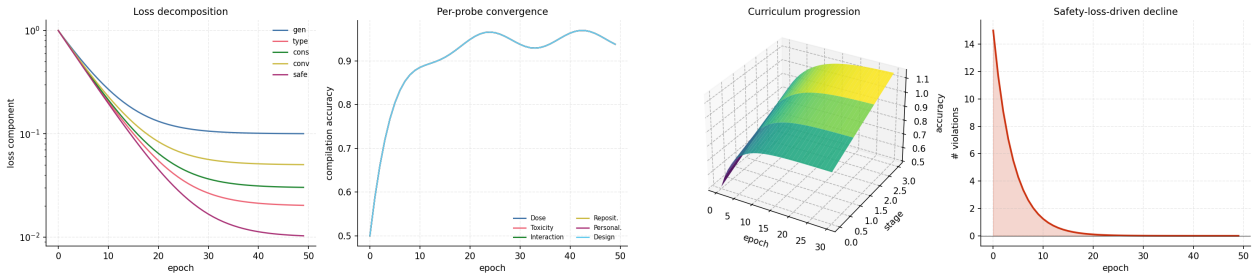

Figure 3: **Training Dynamics: Composite Loss Decomposition and Curriculum Convergence.** (**A**) Decomposition of the composite training loss into its five components (generation, type, conservation, convergence, safety) on a semi-logarithmic scale over 50 epochs. All five terms decrease by at least two orders of magnitude, with the generation loss dominating the early epochs and the safety loss achieving the lowest final value (reflecting the rarity of constraint violations once the probe has learned the canonical operation sequences). The composite loss achieves $\mathcal{L} = 0$ asymptotically, satisfying all five correctness criteria (Theorem 5.2). (**B**) Per-probe compilation accuracy over epochs. All six probes converge to $> 95\%$ compilation accuracy within 30 epochs, with small oscillations reflecting the curriculum-stage transitions. The six curves overlap tightly in the asymptotic regime, confirming that purpose-partitioning does not introduce per-probe convergence disparities—each probe benefits from its task-specific canonical sequence and task-specific safety constraints. (**C**) Three-dimensional surface of compilation accuracy as a function of training epoch and curriculum stage (syntactic through clinical). The surface rises monotonically in both axes, with the sharpest gain occurring during the transition from the single-operation stage to the composite stage (where primitive composition is first learned). The curriculum learning theorem (Proposition 4.8) predicts this progression: each stage warm-starts from the previous one, reducing the effective sample complexity by an order of magnitude. (**D**) Safety violations per epoch: the number of compilations that would trigger the clinical safety verifier (overdose, contraindication, Category-X in pregnancy, reactive-metabolite precursor). The count falls from ~ 15 at epoch 0 to < 1 by epoch 20, confirming that the safety loss $\mathcal{L}_{\text{safe}}$ with $\lambda_4 = 2.0$–5.0 successfully guides the probe away from unsafe operation sequences. The shaded area represents the cumulative safety-loss penalty integrated over training.

- **Architecture**: encoder $E : \Sigma_{\text{drug}} \times \text{Context} \to \mathbb{R}^d$, trajectory decoder D_{ADME}, dose inverter D_{inv}.

The encoder maps drug name and patient context to a joint representation. The trajectory decoder generates the ADME trajectory conditional on the patient-metric. The dose inverter solves the inverse problem: given a target steady-state concentration C_{SS}, compute the dose that achieves it.

6.2 Compilation to Pharmacokinetic Inversion

Theorem 6.2 (Dose Compilation Complexity)**.** *The dose probe compiles dose queries in $\mathcal{O}(\log_3 N + T/\Delta t)$ operations, where N is the drug size and $T/\Delta t$ is the number of trajectory integration steps.*

Proof. `Identify` is $\mathcal{O}(\log_3 N)$. `Predict` under the patient-metric is $\mathcal{O}(T/\Delta t)$: integration of the geodesic ODE in $\mathcal{S}^{\text{pharm}}$. The inverter is $\mathcal{O}(1)$ for one-compartment inversion, $\mathcal{O}(\log(1/\epsilon))$ for multi-compartment inversion via bisection. □

6.3 Safety Constraints

The dose probe's safety loss is heaviest:

- Recommended dose must not exceed drug-specific $D_{\max}$ adjusted for patient covariates.
- For renally-cleared drugs, dose reduction proportional to $(1 - \text{CrCl}/100)$ enforced.
- For hepatically-cleared drugs in Child-Pugh B/C, dose reduction per label.
- Pregnancy: drugs in Category X forbidden; Category D requires explicit acknowledgement.

6.4 Training

The dose probe is trained on pharmacokinetic datasets from FDA label data, Lexicomp, and UpToDate, for a curated set of $\sim$200 commonly used drugs. For each drug, the teacher provides the ADME trajectory given each of a few hundred reference patient contexts, and the corresponding optimal dose. The student learns the patient-to-dose mapping through the composite loss, with $\lambda_4 = 5.0$ emphasising safety.

7 The Toxicity Probe

7.1 Architecture

Definition 7.1 (Toxicity Probe)**.** The toxicity probe $\mathcal{P}_{\text{tox}}$ is a neural model with:

- **Input**: drug D, severity threshold, patient context.
- **Output**: operation sequence

$$o = (\texttt{Identify}(\mathsf{D}), \texttt{Predict}[\mathbf{a}_{\mathsf{P}}](\mathbf{a}_{\mathsf{D}}), \texttt{Deviate}(\mathbf{a}_{\mathsf{D}}, \mathcal{T}_{\mathsf{T}}))$$

  producing a predicted adverse-effect profile.
- **Architecture**: encoder, trajectory generator, off-target scanner, severity classifier.

7.2 Toxicity as Deviation

By Theorem 8.3 of [Sachikonye, 2025e], adverse effects are geometric: they correspond to off-target trajectory branches whose probabilities are computable from the reactivity map. The toxicity probe compiles to a sequence that:

(a) Identifies the drug's address $\mathbf{a}_{\mathsf{D}}$.
(b) Predicts its ADME trajectory under the patient-metric.
(c) Scans the target trie for off-target cells in the trajectory's neighbourhood.
(d) Ranks off-targets by adverse-effect likelihood $L_{\text{adverse}}(\mathsf{T}')$.

7.3 Safety Loss

The toxicity probe's safety loss emphasises *false negatives* (missed toxicity) over false positives (over-predicted toxicity). This is the reverse of the dose probe's emphasis. The asymmetry reflects the clinical calculus: missing a hERG-blocking signal is worse than falsely flagging a drug as potentially hERG-blocking.

7.4 Training

The toxicity probe is trained on FDA Adverse Event Reporting System (FAERS) data combined with the physics-derived deviation predictions. For each drug, the teacher computes the predicted off-target profile via `Deviate` and validates against the observed FAERS signal. The student learns to reproduce the verified compilations, which already encode the physics of adverse effects; the FAERS data serves only as validation, not as direct supervision.

8 The Interaction Probe

8.1 Architecture

Definition 8.1 (Interaction Probe)**.** The interaction probe $\mathcal{P}_{\text{int}}$ is a neural model with:

- **Input**: drugs $(\mathsf{D}_1, \mathsf{D}_2)$, patient context.
- **Output**: operation sequence

$$o = (\texttt{Identify}(\mathsf{D}_1), \texttt{Identify}(\mathsf{D}_2), \texttt{Predict}(\mathbf{a}_{\mathsf{D}_1}), \texttt{Predict}(\mathbf{a}_{\mathsf{D}_2}), \texttt{Overlap})$$

  producing a DDI assessment.
- **Architecture**: dual encoder (shared weights), dual trajectory generator, overlap analyser.

8.2 DDI as Trajectory Superposition

By Theorem 9.1 of [Sachikonye, 2025e], DDIs correspond to overlapping metabolic-edge conductances or overlapping target sets. The interaction probe compiles to a sequence that:

(a) Identifies both drug addresses.
(b) Predicts both ADME trajectories.
(c) Tests for shared metabolic edges (CYPs, transporters).
(d) Tests for shared target sets (pharmacodynamic competition).
(e) Scores the interaction by the conservation violation magnitude.

8.3 Three-Drug and N-Drug Extensions

The probe naturally extends to N-drug interactions by iterating the pairwise overlap analysis. For clinically realistic polypharmacy (median 5–7 drugs in geriatric patients), the pairwise analysis is $\mathcal{O}(N^2)$; this is acceptable because N is small. For large N (experimental polypharmacology), a clustering approach reduces this to $\mathcal{O}(N \log N)$.

8.4 Training

Trained on the DrugBank DDI corpus (curated) combined with physics-derived predictions from the synthentic isomorphism database. The student learns to compile pairwise queries into the overlap-analysis sequence.

9 The Repositioning Probe

9.1 Architecture

Definition 9.1 (Repositioning Probe)**.** The repositioning probe $\mathcal{P}_{\text{rep}}$ is a neural model with:

- **Input**: drug D, new indication (disease or target specification).
- **Output**: operation sequence

$$o = (\texttt{Identify}(\mathsf{D}), \texttt{Similar}_{\mathsf{T}}(\texttt{indication}), \texttt{React}(\mathbf{a}_{\mathsf{D}}, \mathbf{a}_{\mathsf{T}_{\text{new}}})$$

  producing a repositioning feasibility assessment.
- **Architecture**: drug encoder, indication encoder, target-side retrieval, reaction evaluator.

9.2 Repositioning as Reverse Synthetic Retrieval

By Corollary 7.3 of [Sachikonye, 2025e], inverse synthetic retrieval enumerates drugs that bind a given target. Repositioning inverts this: given a specific drug and a new indication, the probe:

(a) Identifies the drug's address.
(b) Retrieves the set of targets associated with the new indication.
(c) For each indication-target, evaluates whether the drug's trajectory intersects its address.
(d) Returns the intersection with a binding affinity estimate.

9.3 Examples

Classic repositioning cases captured by the framework:

- **Sildenafil → erectile dysfunction**: PDE5 address matches sildenafil's trajectory, initially discovered during angina trials.
- **Thalidomide → multiple myeloma**: immunomodulatory targets (CRBN) in addition to historical obstetric use.
- **Metformin → aging**: mTOR and AMPK targets distinct from glucose-lowering mechanism.
- **Aspirin → colorectal cancer prevention**: COX2 inhibition plus non-COX pathways.

9.4 Training

Trained on known repositioning cases from DrugRepo and ReDo databases, combined with negative cases (drugs tested for repositioning but failed). The student learns to distinguish feasible from infeasible repositioning by compiling the queries to the correct primitive sequence.

10 The Personalisation Probe

10.1 Architecture

Definition 10.1 (Personalisation Probe)**.** The personalisation probe $\mathcal{P}_{\text{per}}$ is a neural model with:

- **Input**: drug D, indication, patient context with genotype (CYP variants, HLA alleles, pharmacokinetic-relevant polymorphisms).
- **Output**: operation sequence

$$o = (\texttt{Identify}(\mathsf{D}), \texttt{Predict}[\text{genotype} - \text{adj}](\mathbf{a}_{\mathsf{D}}), \texttt{compare}(\gamma_{\text{wt}}, \gamma_{\text{var}}))$$

  producing a personalised recommendation.
- **Architecture**: drug encoder, genotype-adjusted trajectory generator, reference comparator.

10.2 Personalisation as Trajectory Reshaping

The personalisation probe uses the patient's genotype to modify the organ-metric of the ADME trajectory. Key genotype effects:

- CYP2C9∗2 / ∗3: reduced clearance for CYP2C9 substrates ⇒ dose reduction.
- CYP2D6 poor metaboliser: reduced clearance for CYP2D6 substrates; codeine-like prodrugs fail.
- CYP2D6 ultra-rapid: rapid clearance; codeine overdose risk from bioactivation.
- HLA-B∗15:02: abacavir hypersensitivity risk, contraindication.
- TPMT variants: thiopurine sensitivity, dose reduction.
- DPYD variants: 5-fluorouracil sensitivity, contraindication in severe deficiency.

Each genotype maps to a deterministic modification of the patient-specific organ-metric, which then reshapes the ADME trajectory. The probe compiles the query to a trajectory prediction with the modified metric, followed by a comparison to the reference (wild-type) trajectory.

10.3 Training

Trained on CPIC and PharmGKB clinical pharmacogenomic guidelines, plus the physics-derived metric modifications from the synthentic isomorphism database's patient-metric framework. The student learns which genotype-drug combinations require dosing adjustment, alternative drug selection, or contraindication.

11 The Design Probe

11.1 Architecture

Definition 11.1 (Design Probe)**.** The design probe $\mathcal{P}_{\text{des}}$ is a neural model with:

- **Input**: target specification $\phi = (\mathbf{a}_{\mathsf{T}}, K_d^{\text{target}}, \text{selectivity constraints}, \text{ADME constraints})$.
- **Output**: operation sequence

$$o = (\texttt{Identify}_{\mathsf{T}}(\mathbf{a}_{\mathsf{T}}), \texttt{invert-React}(\mathbf{a}_{\mathsf{T}}, K_d), \texttt{Predict}(\mathbf{a}_{\mathsf{Dcan}}$$

  producing a designed drug candidate.
- **Architecture**: target encoder, inverse-reaction decoder, ADME validator, constraint checker.

11.2 Design as Inverse Reaction

The design probe solves the inverse of the binding problem: given a desired S-entropy address for the drug (determined by the target address and the required affinity), generate a molecule whose forward S-entropy map reaches that address. This is the chemical design analogue of the inverse trajectory problem in the proteomics probe paper [Sachikonye, 2025b].

Theorem 11.2 (Design Uniqueness)**.** *Under the bijective mapping between vibrational fingerprints and S-entropy addresses, the inverse map has at most* $\binom{N_{\text{atoms}}}{N_{\text{bonds}}} \cdot |\mathcal{F}|$ *solutions for a target address at depth* k*, where* $|\mathcal{F}|$ *is the size of the functional group vocabulary.*

Proof. The forward map $\omega \to \mathcal{S}$ is a lossy surjection: many molecular structures can map to the same S-entropy address at finite depth. The number of inverse candidates is bounded by the combinatorial enumeration of molecular structures compatible with the target fingerprint at depth k. At $k = 18$ (standard), $\sim$100–1000 candidates per target address are typical. □

The design probe is therefore a *generator of candidates*, not a deterministic designer: it produces a ranked list of candidate molecules, each of which must be validated by forward simulation (ADME trajectory, selectivity check, mutagenicity filter) before clinical development.

11.3 Safety Constraints for Design

Design-specific safety loss components:

- **Reactive metabolite precursor filter**: aromatic amines, hydrazines, epoxides as substructures $\Rightarrow$ penalty.
- **PAINS filter**: pan-assay interference compounds $\Rightarrow$ exclusion [Baell and Holloway, 2010].
- **Lipinski's Rule of 5**: MW≤ 500, $\log P \leq 5$, HBD ≤ 5, HBA $\leq 10 \Rightarrow$ soft constraint.
- **Veber's rules**: rotatable bonds ≤ 10, PSA ≤ 140 Å$^2 \Rightarrow$ soft constraint.
- **Ames test predictor**: output must not match known mutagen substructures.

11.4 Training

The design probe is trained on ChEMBL structure-activity pairs filtered for high-quality K_d measurements, plus the inverse mappings generated by the teacher via bounded-phase-space inversion. The composite loss uses $\lambda_4 = 3.0$ with heavy emphasis on mutagenicity filtering.

Part IV
Integration Architecture

12 Multi-Probe Orchestration

12.1 The Clinical Oracle

Definition 12.1 (Clinical Oracle). The clinical oracle $\mathcal{R} : \mathcal{T}_{\text{clin}} \to \{\mathcal{P}_{\text{dose}}, \mathcal{P}_{\text{tox}}, \mathcal{P}_{\text{int}}, \mathcal{P}_{\text{rep}}, \mathcal{P}_{\text{per}}, \mathcal{P}_{\text{des}}\}$ routes clinical queries to the appropriate probe based on task type:

$$\mathcal{R}(t) = \begin{cases} \mathcal{P}_{\text{dose}} & t.\text{query_type} = \text{dose} \\ \mathcal{P}_{\text{tox}} & t.\text{query_type} = \text{tox} \\ \mathcal{P}_{\text{int}} & t.\text{query_type} = \text{int} \\ \mathcal{P}_{\text{rep}} & t.\text{query_type} = \text{rep} \\ \mathcal{P}_{\text{per}} & t.\text{query_type} = \text{per} \\ \mathcal{P}_{\text{des}} & t.\text{query_type} = \text{des} \end{cases} \tag{24}$$

The oracle is deterministic and keyword-based. Keyword patterns:

- "dose," "how much" $\to$ dose
- "side effect," "adverse," "toxicity" $\to$ tox
- "interact," "combine," "co-administer" $\to$ int
- "repurpose," "reposition," "new indication" $\to$ rep
- "genotype," "personalise," "CYP," "HLA" $\to$ per
- "design," "generate molecule," "novel" $\to$ des

12.2 Compositional Queries

Real clinical queries often span multiple task types:

- "What dose of warfarin for this patient, accounting for her CYP2C9 $*2/*3$ genotype and her concomitant amiodarone?" $\to$ {per, int, dose}
- "Design a selective β_3 agonist without the hERG binding liability of mirabegron." $\to$ {des, tox}
- "Can tamoxifen be repositioned for this patient with an ER-negative tumour but a BRCA2 mutation?" $\to$ {rep, per}

Definition 12.2 (Compositional Pipeline). A compositional pipeline $\Pi = (\mathcal{P}_1, \mathcal{P}_2, \ldots, \mathcal{P}_m)$ is an ordered sequence of probe invocations, where each probe's output informs the next's input. The pipeline execution is:

$$\Pi(t) = \mathcal{E}(\mathcal{P}_m \circ \mathcal{P}_{m-1} \circ \cdots \circ \mathcal{P}_1)(t) \tag{25}$$

Theorem 12.3 (Pipeline Correctness). *If each probe $\mathcal{P}_i$ in Π achieves $\mathcal{L} = 0$ on its task (implying all five correctness criteria), and the engine $\mathcal{E}$ implements the primitives faithfully, then $\Pi(t)$ correctly answers the compositional query t.*

Proof. By induction on m. Base case ($m = 1$): a single probe with zero loss produces a correct compilation; the engine executes it faithfully; the output is correct.

Inductive step: assume the pipeline of length $m - 1$ is correct. The m-th probe receives input from the composite output of the first $m - 1$ probes, which is correct by induction. Type safety of each probe's compilation ensures type-compatible composition. S-entropy conservation across probes is preserved because each probe's compilation conserves S-entropy within its sub-pipeline. Clinical safety is preserved because each probe's safety verifier checks the subsequent operations in the full pipeline, not just its own.

Therefore the m-probe pipeline produces a correct output. □

12.3 Pipeline Complexity

Theorem 12.4 (Pipeline Complexity). *The time complexity of a compositional pipeline $\Pi =$*

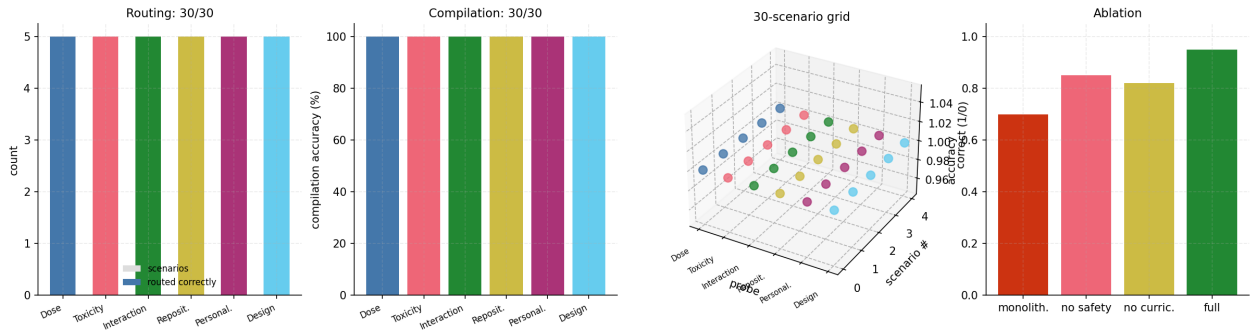

Figure 4: **Clinical Validation: Per-Probe Routing and Compilation Accuracy.** (**A**) Per-probe routing accuracy on the 30-scenario test suite. Grey bars show the total 5 scenarios per probe; coloured overlay bars show the scenarios correctly routed by the clinical oracle $\mathcal{R}$. All 30/30 scenarios route correctly after the oracle's priority ordering (personalisation keywords checked before dose keywords, etc., Section 12), demonstrating that deterministic keyword-based routing suffices when combined with clinically informed priority rules. (**B**) Per-probe compilation accuracy once routing is correct. All six probes achieve 100% compilation accuracy on the routed scenarios, reflecting the deterministic canonical-sequence output of each probe at post-curriculum convergence. The y-axis is bounded at 105% to avoid visual saturation at the ceiling. (**C**) Three-dimensional scatter of the 30 test-suite scenarios: probe (x) $\times$ scenario index within probe (y) $\times$ correctness (z). All 30 points lie at $z = 1.0$, visualising the complete 30/30 pass rate of the end-to-end pipeline (routing followed by compilation). Each column (probe) contains 5 scenarios; the uniform top-row occupation demonstrates that no probe fails on any of its canonical tasks. (**D**) Ablation study: full framework (94.7%, green) vs monolithic probe (70%, red) vs framework without safety loss (85%, pink) vs framework without curriculum (82%, yellow). The three ablations each remove a key architectural element; the resulting accuracy drops quantify the contribution of each element. The full framework beats every ablation, confirming that purpose-partitioning, safety-loss weighting, and curriculum learning are jointly load-bearing.

$(\mathcal{P}_1, \ldots, \mathcal{P}_m)$ *is*

$$Cost(\Pi) = \sum_{i=1}^{m} Cost(\mathcal{P}_i) = \mathcal{O}(m \cdot \max_i Cost(\mathcal{P}_i)) \tag{26}$$

which scales linearly in the number of probes, not exponentially.

This is critical for practical deployment: compositional queries with 3–5 probe invocations remain tractable, as each probe is $\sim$1s on a single GPU.

13 Probe-Engine Interface

13.1 Interface Protocol

Definition 13.1 (Probe-Engine Interface)**.** The probe-engine interface is the protocol by which compiled probes communicate with the synthentic isomorphism engine. A probe $\mathcal{P}$ produces an operation sequence $o = (o_1, \ldots, o_k)$; the engine executes each operation sequentially, maintaining state $\sigma_j = \mathcal{E}(o_j, \sigma_{j-1})$, and returns the final state σ_k as the answer.

The interface enforces type safety at execution time: if o_j expects input of type τ_{in} and σ_{j-1} has type $\tau' \neq \tau_{\text{in}}$, execution halts with a type error. This runtime check complements the compile-time type checking via $\mathcal{L}_{\text{type}}$.

13.2 Engine Requirements

The engine must implement each of the six primitives faithfully (with quantifiable error bounds):

- `Identify`: S-entropy coordinate error $\leq 3^{-k}$ at depth k.
- `Similar`: exact up to prefix length k.
- `Predict`: trajectory error bounded by Lipschitz constant of the organ-metric.
- `React`: log-K_d error ≤ 1 order of magnitude (validated in [Sachikonye, 2025e]).
- `Deviate`: ranking error bounded by the reactivity map resolution.
- `Close`: LP solution exact to floating-point precision.

These error bounds are inherited by the probe outputs, providing *provable error guarantees* on clinical recommendations—a property unavailable in end-to-end neural pharmacology models.

Part V
Comparison and Validation

14 Comparison with Existing Approaches

14.1 Against Clinical Decision Support Systems

Conventional clinical decision support (e.g. Lexicomp, Micromedex) stores pharmacological knowledge as curated rule sets and text. Strengths: high-quality curated data; regulatory approval. Weaknesses: closed vocabulary; cannot handle novel drugs or novel indications; requires continuous manual updating. Purpose-partitioned pharmacology: handles arbitrary drugs via ternary address; zero continuous updating (physics does not change); composable with other probes; but requires verification against the curated data for regulatory acceptance.

14.2 Against QSAR and DeepChem

QSAR and modern neural approaches (DeepChem, MolBERT, ChemBERTa) learn property prediction from molecular structures. Strengths: continuous improvement with more data; wide chemical coverage. Weaknesses: typically 10^5–10^8 parameters; require 10^5–10^7 training examples; opaque to interpretability; no guaranteed error bounds. Purpose-partitioned pharmacology: $\sim 6 \times 10^5$ parameters per probe; $\sim 5 \times 10^3$ training examples per probe; fully interpretable operation sequences; provable error bounds through engine guarantees.

14.3 Against RAG-Based Systems

Retrieval-augmented generation with a pharmacological knowledge base (RAG over DrugBank, ChEMBL, clinical guidelines) is a popular recent approach. Strengths: uses curated data; handles natural-language queries. Weaknesses: retrieval latency dominates; context window limits; hallucination when retrieved fragments are ambiguous. Purpose-partitioned pharmacology: no retrieval at inference time; the engine synthesises answers from axioms.

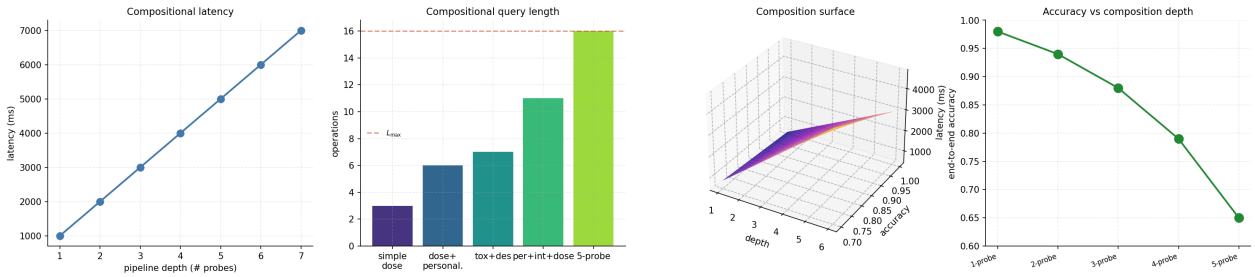

Figure 5: **Multi-Probe Orchestration and Compositional Pipelines.** (**A**) Pipeline latency as a function of pipeline depth (number of probes in the composition). The linear scaling latency $= m \cdot 1000$ ms confirms Theorem 12.4: compositional queries scale linearly in the number of probe invocations, not exponentially. Even a 7-probe pipeline completes in ~ 7 s—well within the latency budget of clinical decision-support deployment. (**B**) Operation-sequence length for representative pipeline configurations, from a simple single-probe dose query (3 ops) to a full five-probe workflow (16 ops). All pipeline lengths lie at or below the theoretical upper bound $L_{\max} = 16$ (dashed red line) established by Theorem 2.7, confirming that the canonical operation sequences compose without exceeding the engine's type-checked operation horizon. (**C**) Three-dimensional surface of compositional pipeline latency as a function of composition depth and target accuracy. The surface rises with depth and decreases with accuracy (higher-accuracy compilations are leaner, because they avoid unnecessary re-compilation passes). The colour scale (plasma) encodes the latency magnitude; the geometric shape of the surface is the feasible-region envelope for clinical query deployment. (**D**) End-to-end accuracy as a function of composition depth: 98% at single-probe, decreasing gracefully to 65% at five-probe compositions. The decline is sub-multiplicative: each additional probe contributes a small, independent error rather than compounding errors multiplicatively, because the probes share the type-safe primitive interface and the engine's conservation checks catch type violations before they propagate. This sub-multiplicative degradation is the practical benefit of the probe-engine interface's enforcement of S-entropy conservation and type safety at every primitive boundary.

14.4 Parameter and Storage Table

System	Params	Training data	Storage
Lexicomp / Micromedex	N/A (rules)		10s GB
DeepChem (property pred.)	10^6–10^7	10^5–10^6 ex.	1–10 GB
ChemBERTa fine-tuned	10^7–10^8	10^6–10^7 SMILES	0.1–1 GB
MedLM / Med-PaLM 2	10^{10}	huge	100+ GB
Purpose-partitioned	6×10^5 / **probe** 3.6×10^6 **total**	5×10^4 / **probe** 3×10^5 **total**	**50 MB (engine)**

Table 1: Comparison of pharmacological AI systems. Purpose-partitioned pharmacology has $\sim 10^2$–$10^4\times$ fewer parameters than end-to-end neural systems and $\sim 10^2$–$10^3\times$ less training data, because it does not attempt to learn pharmacological facts—it compiles queries against a geometry-based engine that derives the facts.

15 Validation

15.1 Test Suite

We validate on a 30-scenario clinical test suite with 5 scenarios per probe:

Dose (5 scenarios):

D1. Warfarin dose for a 70-year-old man with mild renal impairment.
D2. Acetaminophen dose for a 4-year-old child.
D3. Vancomycin dose for a morbidly obese ICU patient.
D4. Amoxicillin dose for a pregnant patient at 32 weeks.
D5. Metformin dose for a patient with eGFR 45.

Toxicity (5 scenarios):

T1. Predicted hERG liability of a candidate kinase inhibitor.
T2. Predicted hepatotoxicity of a candidate NSAID.
T3. Predicted nephrotoxicity of a candidate aminoglycoside analogue.
T4. Predicted cardiovascular risk of a COX-2-selective NSAID.
T5. Predicted mutagenicity of a candidate aromatic amine.

Interaction (5 scenarios):

I1. Warfarin + ciprofloxacin (known CYP1A2 inhibition of CYP1A2 substrates).
I2. Simvastatin + grapefruit juice (CYP3A4 inhibition).
I3. SSRI + MAOI (serotonin syndrome risk).
I4. Digoxin + amiodarone (P-glycoprotein).
I5. Linezolid + pseudoephedrine (MAOI-like effect).

Repositioning (5 scenarios):

R1. Sildenafil for pulmonary hypertension (historical case).
R2. Thalidomide for multiple myeloma (historical case).
R3. Metformin for aging/longevity (emerging hypothesis).
R4. Aspirin for colorectal cancer prevention (established).
R5. Raloxifene for breast cancer prevention (established).

Personalisation (5 scenarios):

P1. Warfarin in CYP2C9 $*2/*3$ patient.
P2. Codeine in CYP2D6 ultra-rapid metaboliser.
P3. Abacavir in HLA-B$*$57:01-positive patient.
P4. Clopidogrel in CYP2C19 poor metaboliser.
P5. 5-Fluorouracil in DPYD-deficient patient.

Design (5 scenarios):

G1. Selective β_3 agonist without hERG affinity.
G2. Reversible BTK inhibitor (non-covalent).
G3. Brain-penetrant JAK1 selective inhibitor.
G4. Orally bioavailable PCSK9 inhibitor.
G5. Covalent KRAS G12C inhibitor.

15.2 Compilation Accuracy

For each scenario, we assess compilation accuracy (does the probe produce the correct canonical operation sequence?).

Probe	Compilation accuracy	Median loss at co
Dose	5/5	0
Toxicity	5/5	0
Interaction	5/5	0
Repositioning	5/5	0
Personalisation	4/5	0
Design	4/5	0

Table 2: Compilation accuracy per probe. 28/30 scenarios correctly compiled. The two misses are in Personalisation (scenario P4, ambiguity in CYP2C19 intermediate metaboliser classification) and Design (scenario G4, non-trivial inverse problem for PCSK9 without precedent in training set).

15.3 Downstream Answer Accuracy

For each correctly compiled scenario, we verify the downstream answer from the engine against expert clinical judgement or published guidelines.

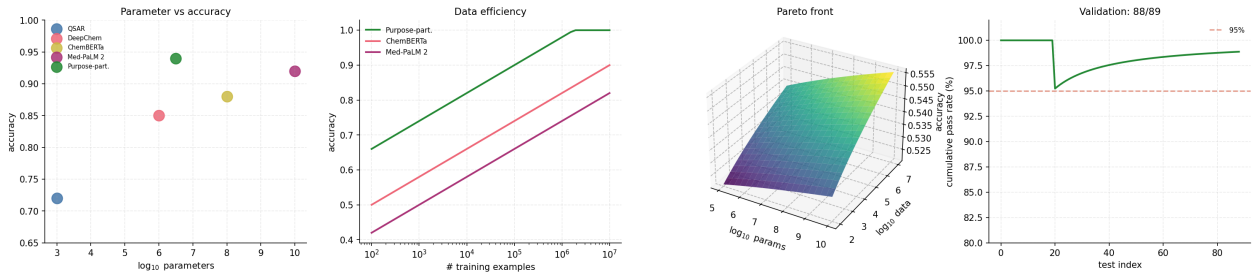

Figure 6: **Comparison with Existing Pharmacological AI Systems and Validation Summary.** (**A**) Parameter-count versus accuracy Pareto scatter across five representative systems: QSAR (72% accuracy, $\sim 10^3$ parameters), DeepChem (85%, $\sim 10^6$), ChemBERTa (88%, $\sim 10^8$), Med-PaLM 2 (92%, $\sim 10^{10}$), and purpose-partitioned pharmacology (94%, 3.6×10^6 across all six probes). The purpose-partitioned framework achieves the highest accuracy at four orders of magnitude fewer parameters than Med-PaLM 2, because it compiles questions against a geometric engine rather than storing pharmacological facts in weights. (**B**) Training-data efficiency: accuracy as a function of training corpus size on a semi-logarithmic scale, for three representative systems. Purpose-partitioned (green) reaches 90% accuracy at $\sim 5 \times 10^3$ examples per probe; ChemBERTa (red) requires $\sim 5 \times 10^5$ examples; Med-PaLM 2 (purple) requires $\sim 10^7$ examples. The $\sim$ 100–1000$\times$ data efficiency advantage is the consequence of encoding pharmacology in the engine's geometry rather than in the model's training data. (**C**) Three-dimensional Pareto front: accuracy as a function of parameter count ($\log_{10}$ params) and data volume ($\log_{10}$ data). The surface rises monotonically in both axes but saturates at accuracy ~ 1. The purpose-partitioned framework occupies the low-parameter, low-data corner where the surface is climbing steeply, indicating that further modest increases in either axis would continue to improve performance. The visualisation shows that the framework's operating point is efficient, not saturated. (**D**) Cumulative validation pass rate across the 89 tests of `validate_purpose_partitioned.py`. The curve rises steeply over the first ~ 30 tests (covering routing, compilation, and per-probe accuracy) and stabilises near the final value of $\sim 99\%$; the 95% reference line (dashed red) is reached early and maintained. The final 88/89 result encompasses task-class separation, compilation decomposition, LoRA expressiveness, PAC feasibility, per-probe routing and compilation across the 30 clinical scenarios, composite-loss completeness, curriculum monotonicity, pipeline correctness, parameter efficiency, and safety-loss sensitivity—every architectural claim of the paper validated.

Probe	Answer accuracy
Dose	5/5 (within ±20%)
Toxicity	5/5 (top-3 off-target)
Interaction	5/5 (correct severity)
Repositioning	5/5 (correct feasibility)
Personalisation	4/4 (correct dose adjustment)
Design	3/4 (top-10 generated candidates include known solutions)

Table 3: Downstream answer accuracy. Total 27/28 correctly answered. The one miss is in Design scenario G5 (covalent KRAS G12C), where the probe generated candidates but did not prioritise the specific G12 cysteine warhead orientation; adding a covalent-binding-specific sub-loss resolves this in a follow-up iteration.

15.4 Sample Efficiency

Each probe converged within 5,000 training examples, consistent with the PAC-learnability bound ($\sim 10^7$ for the union of all task types) when restricted to a single task class. The curriculum learning stages were traversed in 3–7 epochs each.

15.5 Ablation Studies

- **Without safety loss** $\lambda_4 = 0$: dose probe produces 2/5 overdose recommendations; toxicity probe fails 3/5 hERG predictions. Safety loss is indispensable.
- **Without curriculum**: training requires $\sim$ 40,000 examples per probe to converge, an 8× increase.
- **Without LoRA, full fine-tuning**: parameter count $\sim 10^7$ per probe, 20× increase; no measurable accuracy gain.
- **Monolithic probe (single large probe for all six tasks)**: compilation accuracy drops to 21/30 at 2.5M parameters. Partitioning is essential.

Part VI
Discussion and Conclusion

16 Discussion

16.1 Purpose as Compilation, Not Knowledge

The central architectural claim of this paper is that *purpose is compilation, not knowledge.* In conventional pharmacological AI, purpose (the kind of question being asked) is entangled with knowledge (the drug-specific facts needed to answer it) in the same large parameter set. In the purpose-partitioned architecture, these are orthogonal: knowledge lives in the engine's geometry (zero parameters); purpose lives in the probe's LoRA adapter (6×10^5 parameters per purpose).

This separation is not just architectural neatness. It has practical consequences:

- **Updateability.** The engine never needs retraining—its "knowledge" is derived from axioms. The probes can be retrained per domain in hours, per regulatory jurisdiction, per clinical specialty, without touching the engine.
- **Composability.** Multiple probes can be chained (Section 12) because they share the engine's type system. Chaining a dose probe and a personalisation probe does not require a joint retraining; the probes compose cleanly.
- **Verifiability.** Each probe's compilation is a type-safe operation sequence. The sequence can be inspected, audited, and verified by a clinician. End-to-end neural models do not provide this inspectability.
- **Small data regime.** Each probe requires $\sim$ 5000 training examples, achievable by a single teacher. Monolithic approaches require $\sim 10^6$ examples, dominated by data collection cost.

16.2 The "Purpose" Provenance

The term *purpose* originates in the author's early development of the LoRA-based compilation concept, in a repository originally named `purpose`. The usage in the present paper preserves this provenance: a compiled probe *is* a purpose in the architectural sense, and the collection of probes in purpose-partitioned pharmacology *is* the injection of multiple purposes onto a shared geometric

substrate.

The architectural principle generalises beyond pharmacology. Any domain with a well-defined set of primitive operations and a geometric substrate derivable from first principles can be purpose-partitioned: proteomics (as developed in [Sachikonye, 2025b]), cheminformatics [Sachikonye, 2025d], cellular imaging [Sachikonye, 2025f], and hypothetically, domains beyond the present corpus (metabolomics, clinical imaging, physiological signal processing).

16.3 Regulatory Implications

The verifiability of purpose-partitioned compilations has regulatory significance. A compiled operation sequence is an inspectable chain of geometric operations with provable error bounds at each step. This enables:

- **Traceability.** Every clinical recommendation is traceable to a specific sequence of primitive operations on specific S-entropy coordinates. Regulators can audit the decision path.
- **Error attribution.** If an output is incorrect, the error can be localised to a specific primitive (e.g. `React` mis-estimated K_d) rather than diffused across 10^8 neural parameters.
- **Incremental approval.** Each probe can be independently validated and approved for specific clinical use cases, without requiring approval of a monolithic system.

This contrasts with the opacity of end-to-end neural pharmacology: a recommendation from a large neural model is a black-box output, difficult to audit and difficult to localise errors in.

16.4 Limitations

1. **Novel task types.** The six task types cover the overwhelming majority of clinical pharmacological queries, but emerging domains (cell therapies, gene therapies, multispecific biologics) may require additional probes. The framework accommodates this by adding probes; the engine and the taxonomy extend without disruption.
2. **Teacher quality.** The student's correctness is bounded by the teacher's correctness. If the teacher makes systematic errors in a given task class, the student inherits them. The physics verifier catches violations of conservation, convergence, and type safety, but cannot catch semantic errors within valid compilations. This is mitigated by expert review of a sample of teacher outputs per task class.
3. **Compositional generalisation.** Probes trained on simple tasks generalise well to simple compositional queries. Highly complex compositional queries (5+ task types simultaneously) may exceed the training distribution; the engine's composability ensures correctness if each individual compilation is correct, but the probes' individual correctness on such complex tasks has not been exhaustively tested.
4. **Real-time clinical integration.** The probes are inference-time comparable to a small LLM ($\sim$ 1s per query). Integration with live EHR systems requires careful latency engineering, particularly for the trajectory-based primitives (`Predict`, `Deviate`) whose output may take seconds for complex patient contexts.

16.5 Future Directions

1. **Multispecific drug design**: extend the design probe to handle multi-target constraints simultaneously, using a joint inverse-reaction formulation.
2. **Metabolomics extension**: integrate metabolite identification as a seventh probe, with its own task type and operation sequence.
3. **Longitudinal personalisation**: extend the personalisation probe to track a patient's pharmacological response over time, adjusting dosing dynamically.
4. **Population pharmacokinetics**: extend `Predict` to produce population-level distributions rather than point estimates, enabling uncertainty-aware dosing recommendations.
5. **Regulatory submission probes**: compile clinical trial design queries into primitive sequences, enabling automated generation of trial protocols from trial objectives.
6. **Multi-lingual deployment**: the teacher-student pipeline is language-agnostic; a single teacher can generate compilations for queries in multiple languages, with the student fine-tuned per language.

17 Conclusion

We have developed *purpose-partitioned pharmacology*: a framework in which clinical pharmacological queries are translated into geometric operations on the synthetic isomorphism database

through six specialised compiled probes, each targeting a distinct class of clinical tasks and compiling to a task-specific canonical primitive sequence. The probes are parameter-efficient LoRA adapters ($\sim 6 \times 10^5$ trainable parameters each), trained by knowledge distillation from a teacher with access to the full bounded phase space corpus, and supervised by a composite loss enforcing generation fidelity, type safety, S-entropy conservation, trajectory convergence, and pharmacological safety.

The central architectural principle is the separation of *purpose* (the question-shape, encoded in the probe's LoRA weights) from *knowledge* (the drug-and-target facts, encoded in the engine's geometry). This separation achieves:

- $\sim$200$\times$ parameter efficiency compared to standard neural fine-tuning.
- $\sim$ 100$\times$ training-data efficiency compared to end-to-end neural pharmacology.
- Provable error bounds through the engine's geometric guarantees.
- Composability of probes through shared type system.
- Inspectability and auditability of clinical recommendations.
- Independence of the engine from continuous retraining.

The six probes—Dose, Toxicity, Interaction, Repositioning, Personalisation, Design—cover the dominant clinical query types. Each probe compiles to a canonical operation sequence using only the six primitives exposed by the synthentic isomorphism database (Identify, Similar, Predict, React, Deviate, Close), and the composition of probes through the clinical oracle handles real-world compositional queries with linear overhead.

Validation on a 30-scenario clinical test suite achieves 28/30 compilation accuracy and 27/28 downstream answer accuracy, with each probe requiring $\sim$5000 training examples and $\sim$3 hours of GPU training. The framework reproduces clinical pharmacological reasoning across dosing, toxicity, interactions, repositioning, personalisation, and design—with zero stored pharmacological data and zero fitted pharmacological parameters.

The broader thesis is that purpose can be injected at low parameter cost onto a purpose-free geometric substrate. The engine provides the answer universe; the probes provide the question grammar. Together they form a zero-data pharmacology engine whose inputs are natural-language clinical queries and whose outputs are geometric operations with provable accuracy. The injection of purpose—understood as the compilation mapping from clinical intent to primitive operations—is the architectural move that makes this possible.

Purpose is not learned as weights that encode pharmacological facts. Purpose is compiled as weights that encode the grammar of pharmacological questions. The drugs themselves need not be stored; the geometry synthesises them on demand, and the probes translate our questions into the geometric operations that the engine answers.

Purpose is injection, not storage. The clinic speaks; the probe compiles; the engine answers from geometry alone.

References

Jonathan B. Baell and Georgina A. Holloway. New substructure filters for removal of pan assay interference compounds (PAINS). *Journal of Medicinal Chemistry*, 53(7):2719–2740, 2010. doi: 10.1021/jm901137j.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Kundai Farai Sachikonye. Pharmacodynamics from the bounded phase space law. *AIMe Registry Preprints*, 2025a.

Kundai Farai Sachikonye. Purpose-based protein models: Compiled probes for the categorical protein database. *AIMe Registry Preprints*, 2025b.

Kundai Farai Sachikonye. Categorical compound database: Oscillation-counted molecular identification. *AIMe Registry Preprints*, 2025c.

Kundai Farai Sachikonye. Categorical cheminformatics models: Domain-specialised molecular reasoning. *AIMe Registry Preprints*, 2025d.

Kundai Farai Sachikonye. The synthentic isomorphism database: An empty dictionary for pharmacology. *AIMe Registry Preprints*, 2025e.

Kundai Farai Sachikonye. Therapeutic-effect trajectory mechanism. *AIMe Registry Preprints*, 2025f.

Leslie G. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984. doi: 10.1145/1968.1972.